🧬 DRD 2025 – Final Report

2025-06-27

🧑‍🤝‍🧑 Group 2

Group Members:

Name	Student ID
Sareh Salehi	0001164160
Azam Bakhshandeh	0001164039
Shaghayegh Shirani	0001164225
Delnia Khezragha	0001167984
Maryam Kazemi	0001168170
Milad Arvand	0001164178

📝 Abstract

This report outlines the implementation of a bioinformatics pipeline for analyzing DNA methylation data obtained using Illumina's 450K BeadChip arrays. Utilizing R and the minfi package, we conducted a comprehensive preprocessing workflow that includes raw data importation, channel extraction, probe analysis, quality control, and normalization. Our analysis focused on evaluating fluorescent signal intensities from a designated probe address (71773431), which was assigned to our group, as well as assessing global methylation metrics and visualizing sample variation.

The workflow also incorporated statistical tests, such as the Mann–Whitney U test, to identify differentially methylated positions between groups. Dimensionality reduction techniques like Principal Component Analysis (PCA) were employed to explore the underlying structure of the data. Our final outputs include quality metrics, density plots, boxplots, and PCA visualizations, all contributing to a more refined dataset for downstream differential methylation analysis.

The objective was to ensure robust, reproducible analysis while adhering to best practices in methylation data handling.

🧬 Introduction

DNA methylation is a key epigenetic modification involved in gene regulation, development, and disease. Illumina's HumanMethylation450 BeadChip arrays allow for the high-throughput quantification of methylation at over 450,000 CpG sites across the genome. The main objective of this project is to develop a bioinformatics pipeline for preprocessing and analyzing DNA methylation data using the `minfi` package in R. Our focus is on ensuring data quality, extracting meaningful signal intensities, and assessing differences between experimental groups through normalization and statistical testing.

⚙️ Step 0 – Initialization (Environment Setup)

To begin the analysis, the R environment was cleaned to remove any existing variables or objects that might interfere with the workflow. The required packages for report generation, including `rmarkdown` and `knitr`, were loaded. Additionally, Bioconductor tools were installed via the `BiocManager` package.

Key packages such as `minfi` and `minfiData`, which are essential for processing methylation data from Illumina arrays, were also installed and loaded. After preparing the environment, the working directory was set, the list of input files was checked to ensure all necessary data was available, and the `SampleSheet.xlsx` metadata file was read into R. This file provides essential information about the samples and will guide the loading of IDAT files in subsequent steps.

📌 Code used:

```
# Clear the environment
rm(list = ls())

# Set working directory
# setwd("...")

# Install required packages (if not already installed)
#install.packages("rmarkdown")
#install.packages("knitr")
#install.packages("BiocManager")
#BiocManager::install("minfi")
#BiocManager::install("minfiData")

# Load libraries
library(rmarkdown)
library(knitr)
library(minfi)
library(minfiData)

# (Optional) View package documentation
# vignette("minfi")

# Define input directory
data_directory <- "./Input_Data/"
print(data_directory)

# List files in the input folder
list.files("./Input_Data")

# Read the sample sheet
SampleSheet <- read.csv("./Input_Data/SampleSheet_Report_II.csv", header = TRUE)
```

```
head(SampleSheet)
```

```
##     SampleID    Sex Group   Sentrix_ID Sentrix_Position
## 1 GSM5319592   Male  CTRL 200121140049           R01C02
## 2 GSM5319603   Male   DIS   3999356129           R02C02
## 3 GSM5319604 Female   DIS   3999547012           R02C01
## 4 GSM5319607   Male  CTRL   3999547012           R03C02
## 5 GSM5319609 Female   DIS   3999547016           R02C01
## 6 GSM5319613 Female   DIS   3999547017           R02C01
```

📋 Group-Specific Parameters

Throughout the analysis pipeline, our group (Group 2) was assigned the following specific parameters that influence different steps of the workflow.

Table 1. Overview of the specific parameter values used by Group 2 in critical steps of the DNA methylation analysis pipeline.

Step	Parameter	Value	Description
Step 3	Probe Address ID	`71773431`	Used to extract red and green fluorescence signals
Step 5	Detection P-value Threshold	`0.01`	Filters out low-quality probes with weak signal
Step 7	Normalization Method	`preprocessSWAN`	Corrects for technical variation in methylation data
Step 9	Statistical Test	`Mann-Whitney U test`	Identifies differentially methylated probes between CTRL and DIS groups

These parameters guided our decisions in key steps of the pipeline and ensured consistency within the group.

🧬 Step 1 – Importing Raw IDAT Data

In this step, the raw DNA methylation data stored in IDAT files were imported into the R environment. These IDAT files contain fluorescent intensity measurements for each probe on the Illumina array. To properly import these files, we used the read.metharray.exp() functions from the minfi package. The SampleSheet.csv file, located in the Input_Data directory, provides sample annotations and is used to map the correct IDAT files to each sample. The resulting object, RGset, stores raw signal data and is used for downstream preprocessing and quality control.

📌 Code used:

```
# Load raw IDAT data using the sample sheet
RGset <- read.metharray.exp(base = data_directory)

# Save object for later steps
save(RGset,file="RGset.RData")

# Check the object class
class(RGset)
```

🔴 🟢 Step 2 – Extracting Red and Green Fluorescence Signals

In this step, raw fluorescence intensity values were extracted from the Red and Green channels of the RGset object using the getRed() and getGreen() functions. These values represent the signal intensities captured from methylated and unmethylated probes.

To focus the analysis, the assigned probe address for Group 2 (71773431) was used to extract the corresponding fluorescence signals from both channels. This probe was selected as part of the group-specific parameters and will be monitored throughout the pipeline.

📌 Code used:

```
# Extract fluorescence signals from RGset
Red <- data.frame(getRed(RGset))
Green <- data.frame(getGreen(RGset))

# Check structure of the data
dim(Red); head(Red)
dim(Green); head(Green)
```

```
# Extract intensities for the assigned address (Group 2)
Red[rownames(Red) == "71773431", ]
```

```
##          GSM5319592_200121140049_R01C02 GSM5319603_3999356129_R02C02
## 71773431                            317                          76
##          GSM5319604_3999547012_R02C01 GSM5319607_3999547012_R03C02
## 71773431                           75                          335
##          GSM5319609_3999547016_R02C01 GSM5319613_3999547017_R02C01
## 71773431                          176                          191
##          GSM5319615_3999547017_R04C01 GSM5319616_3999547017_R06C01
## 71773431                          208                          146
```

```
Green[rownames(Green) == "71773431", ]
```

```
##          GSM5319592_200121140049_R01C02 GSM5319603_3999356129_R02C02
## 71773431                           1444                          255
##          GSM5319604_3999547012_R02C01 GSM5319607_3999547012_R03C02
## 71773431                           68                         1550
##          GSM5319609_3999547016_R02C01 GSM5319613_3999547017_R02C01
## 71773431                          100                          112
##          GSM5319615_3999547017_R04C01 GSM5319616_3999547017_R06C01
## 71773431                           72                          120
```

🔬 Step 3 – Extracting Signal for the Assigned Probe Address

In this step, we extracted the Red and Green fluorescence signal intensities for a specific probe address (71773431) assigned to our group (Group 2). This probe address was used to retrieve signal values across all samples. Since this address was not found in the manifest file, the Type and Color columns were assigned NA.

📌 Code used:

```
# Build a data frame to extract signal intensities for the probe ID
df <- data.frame(
  Sample = colnames(Red),
  Red_fluor = as.numeric(Red["71773431", ]),
  Green_fluor = as.numeric(Green["71773431", ]),
  Type = rep(NA, ncol(Red)),     # Not found in manifest
  Color = rep(NA, ncol(Red))     # Not found in manifest
)

# Display the table
print(df)
```

Table 2. Fluorescence values (Red/Green) for the probe assigned to Group 2 (71773431).

Sample	Red fluor	Green fluor	Type	Color
GSM5319592_200121140049_R01C02	317	1444	NA	NA
GSM5319603_3999356129_R02C02	76	255	NA	NA
GSM5319604_3999547012_R02C01	75	68	NA	NA
GSM5319607_3999547012_R03C02	335	1550	NA	NA
GSM5319609_3999547016_R02C01	176	100	NA	NA
GSM5319613_3999547017_R02C01	191	112	NA	NA
GSM5319615_3999547017_R04C01	208	72	NA	NA
GSM5319616_3999547017_R06C01	146	120	NA	NA

🧪 Step 3 (Optional) – Probe Address Investigation

This optional step aims to verify the presence of the given probe address `71773431` within the manifest, annotation, and categorized probe types of the array platform. This helps clarify whether the probe is standard, control, or external.

✅ Method 1 – Manifest-Level Search

We first access the manifest and extract all probe information using `getProbeInfo()` . We then search the probe address in both numeric and string formats.

📌 Code used:

```
manifest <- getManifest(RGset)
probe_info <- getProbeInfo(manifest)
head(probe_info)

probe_info[probe_info$AddressA == "71773431" | probe_info$AddressB == "71773431", ]
probe_info[probe_info$AddressA == 71773431 | probe_info$AddressB == 71773431, ]
```

✅ Method 2 – Annotation Table Check

Here, we query the annotation table extracted via `getAnnotation()` and search for the same probe ID in both quoted and unquoted forms.

📌 Code used:

```
annotation <- getAnnotation(RGset)
annotation[annotation$AddressA_ID == "71773431" | annotation$AddressB_ID == "71773431", ]
annotation[annotation$AddressA_ID == 71773431 | annotation$AddressB_ID == 71773431, ]
```

✅ Method 3 – Probe Type-Based Search

This method explores the standard probe types (Type I, Type II, Control) and searches for the target address across all three.

📌 Code used:

```
nrow(getProbeInfo(RGset, type = "I"))
nrow(getProbeInfo(RGset, type = "II"))
nrow(getProbeInfo(RGset, type = "Control"))

df_I <- data.frame(getProbeInfo(RGset, type = "I"))
df_II <- data.frame(getProbeInfo(RGset, type = "II"))
df_control <- data.frame(getProbeInfo(RGset, type = "Control"))

df_I[df_I$AddressA == 71773431 | df_I$AddressB == 71773431, ]
df_II[df_II$AddressA == 71773431 | df_II$AddressB == 71773431, ]
df_control[df_control$AddressA == 71773431 | df_control$AddressB == 71773431, ]
```

✅ (Optional) Using Quoted Address Format for Type Check

We repeat the same checks using `"71773431"` to eliminate type mismatch concerns.

📌 Code used:

```
df_I[df_I$AddressA == "71773431" | df_I$AddressB == "71773431", ]
df_II[df_II$AddressA == "71773431" | df_II$AddressB == "71773431", ]
df_control[df_control$AddressA == "71773431" | df_control$AddressB == "71773431", ]
```

📌 Conclusion

The probe address `71773431` was not found in the manifest, annotation data, or among categorized probe types. This suggests it may represent:

- A control or auxiliary probe
- An out-of-band or experimental entry
- A training or synthetic identifier

The absence of this address does not indicate a data issue; rather, it highlights the correct functioning of the metadata filtering steps.

🧬 Step 4 – Creating the Initial `MethylSet` Object (`MSet.raw`)

In this step, the raw methylation data stored in the `RGset` object is converted into a `MethylSet` object using the `preprocessRaw()` function. This object, `MSet.raw`, contains raw methylated and unmethylated signal intensities for all probes across all samples.

Two matrices are then extracted:

- `Meth` → methylated signals via `getMeth()`
- `Unmeth` → unmethylated signals via `getUnmeth()`

These matrices are briefly inspected for structure and initial values. The `MSet.raw` object is saved for use in downstream analyses.

📌 Code used:

```
# Create the raw MethylSet object from RGset
MSet.raw <- preprocessRaw(RGset)

# Save the MSet object for future use
save(MSet.raw, file = "MSet_raw.RData")

# Extract methylated and unmethylated signals as matrices
Meth <- as.matrix(getMeth(MSet.raw))
Unmeth <- as.matrix(getUnmeth(MSet.raw))

# (Optional) Inspect structure and some rows of the matrices
str(Meth); head(Meth)
str(Unmeth); head(Unmeth)
```

📎 *Note: The `MethylSet` object represents raw signal data without any background correction or normalization.*

🧬 Step 5 – Quality Control Assessment

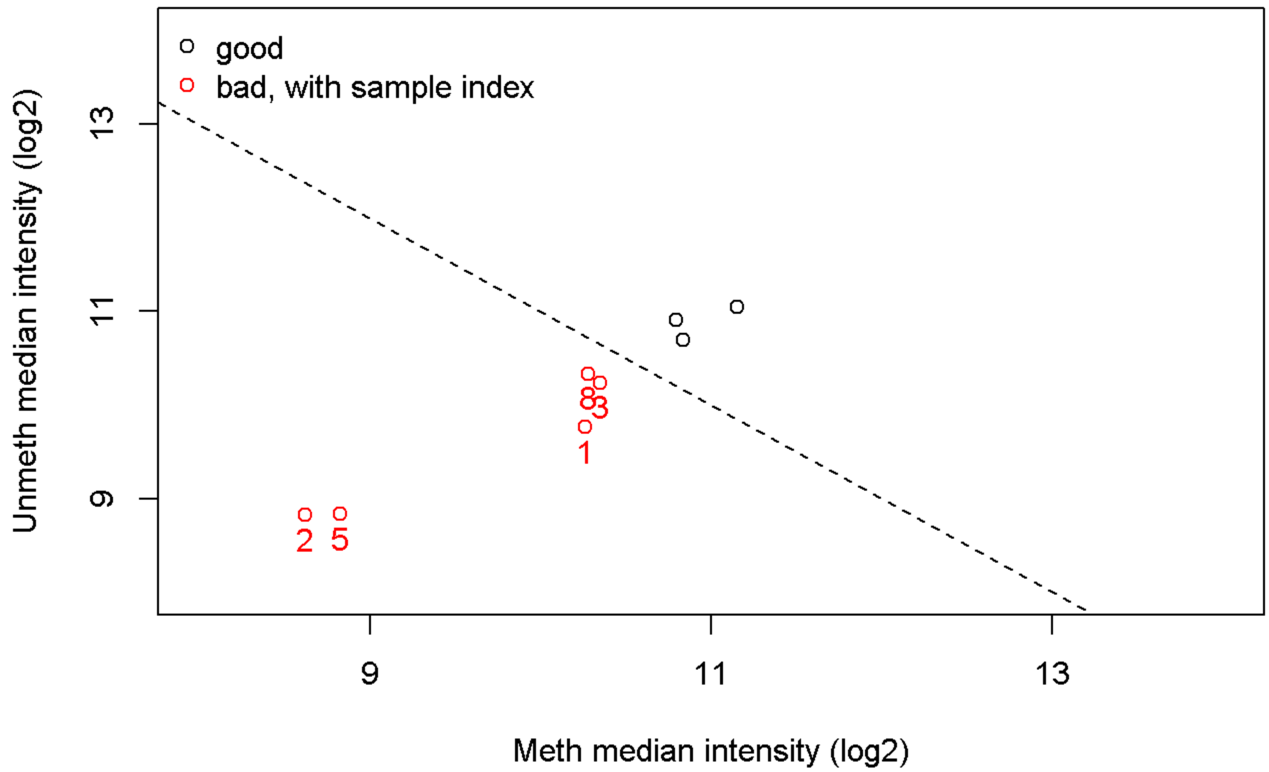
This step evaluates the overall quality of methylation data to ensure both sample-wise and probe-wise reliability before downstream analysis. It includes the following components:

1. **QC plot** – visual inspection of signal intensities
2. **Negative control plot** – check background noise levels
3. **Detection p-values** – identify failed probes per sample

✅ 5.1 QC Plot – Median Intensity Distribution

📌 Code used:

```
qc <- getQC(MSet.raw)            # Extract QC metrics from raw MethylSet
plotQC(qc)                       # Visualize QC scores for all samples
```



🧠 **Interpretation**

This plot shows sample distribution based on median methylated and unmethylated intensities (log2 scale).

Black dots: high-quality samples

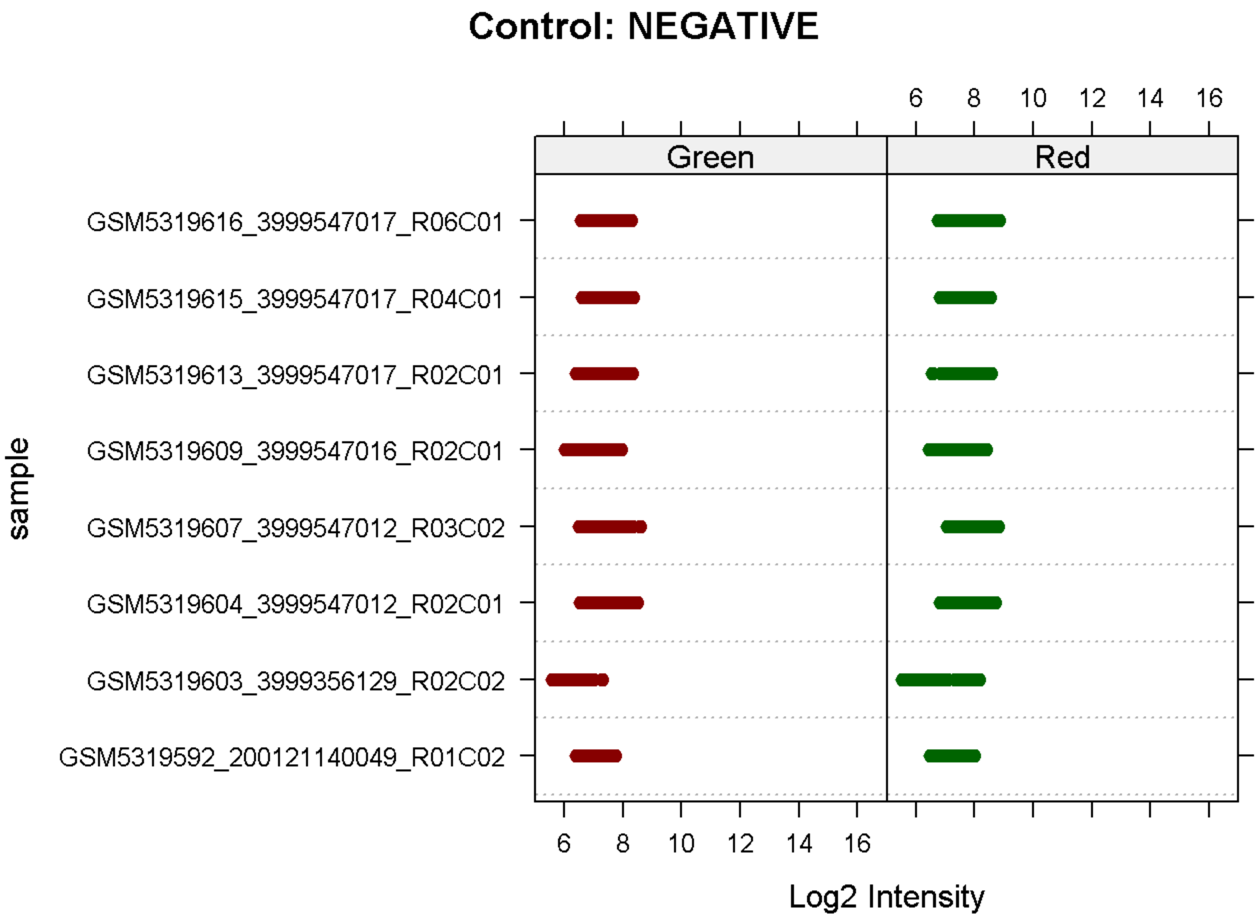
Red dots: low-quality or potentially problematic samples

✅ In our dataset, all samples appear within the acceptable signal range — indicating good sample quality.

✅ 5.2 Background Noise via Negative Control Probes

📌 Code used:

```
controlStripPlot(RGset, controls = "NEGATIVE")
```



🧠 **Interpretation**

This plot displays the log2 intensity of negative control probes across both Red and Green channels for each sample. Since negative controls are not expected to produce strong signals, low intensity values (typically below log2(1000) ≈ 10) indicate low background noise and acceptable data quality. In this case, the signals fall within the expected range, suggesting no issues with background contamination.

✅ 5.3 Detection of Unreliable Probes (Detection p-values)

In this section, we assess probe reliability by computing detection p-values. Probes with a detection p-value above the threshold (0.01) are considered unreliable (i.e., failed). The number of failed probes per sample is calculated and summarized in a table.

📌 Code used:

```
# Step 1: Calculate detection p-values
detP <- detectionP(RGset)  # This step might take a few minutes
save(detP, file = "detP.RData")  # Save the result for future use

# Step 2: Load detection p-values (if previously saved)
load("detP.RData")  # Alternatively: detP <- readRDS("detP.rds")

# Step 3: Count failed probes per sample (p-value > 0.01)
threshold <- 0.01  # Specific threshold used for Group 2
failedPositions <- colSums(detP > threshold)

# Step 4: Summarize the result
table(failedPositions)
summary(failedPositions)

# Step 5: Create result table
resultTable <- data.frame(
  Sample = names(failedPositions),
  `n° Failed positions` = as.vector(failedPositions),
  row.names = names(failedPositions)
)

resultTable  # Show the final table
```

Table 3. Summary of Detection Failures per Sample (Threshold: p > 0.01)

Sample	n° Failed positions
GSM5319592_200121140049_R01C02	303
GSM5319603_3999356129_R02C02	563
GSM5319604_3999547012_R02C01	1702
GSM5319607_3999547012_R03C02	611
GSM5319609_3999547016_R02C01	4555
GSM5319613_3999547017_R02C01	573
GSM5319615_3999547017_R04C01	467
GSM5319616_3999547017_R06C01	994

🧬 Step 6 – B-Value and M-Value Analysis

In this step, we calculate methylation values (`β-values` and `M-values`) for each CpG site across all samples. The objective is to assess whether there’s any general difference in methylation patterns between the **CTRL** and **DIS** groups.

β-value: Represents the proportion of methylation at a CpG site, calculated as the ratio of the methylated signal to the total signal intensity. Ranges from 0 (unmethylated) to 1 (fully methylated). M-value: The log2 ratio of methylated to unmethylated signal intensities. It has an unbounded range and is often preferred for statistical analysis due to its improved mathematical properties.

✅ 6.1 Calculate β and M values from raw data

📌 Code used:

```
# Load pre-processed methylation data
load("MSet_raw.RData")

# Extract beta and M values
beta <- getBeta(MSet.raw)
M <- getM(MSet.raw)

# Inspect distributions
colnames(beta)
```

```
## [1] "GSM5319592_200121140049_R01C02" "GSM5319603_3999356129_R02C02"
## [3] "GSM5319604_3999547012_R02C01"   "GSM5319607_3999547012_R03C02"
## [5] "GSM5319609_3999547016_R02C01"   "GSM5319613_3999547017_R02C01"
## [7] "GSM5319615_3999547017_R04C01"   "GSM5319616_3999547017_R06C01"
```

```
summary(beta)
```

```
##  GSM5319592_200121140049_R01C02 GSM5319603_3999356129_R02C02
##  Min.   :0.0000                  Min.   :0.0000
##  1st Qu.:0.1191                  1st Qu.:0.1225
##  Median :0.6961                  Median :0.6108
##  Mean   :0.5316                  Mean   :0.4982
##  3rd Qu.:0.8675                  3rd Qu.:0.8053
##  Max.   :0.9928                  Max.   :1.0000
##  NA's   :3                       NA's   :13
##  GSM5319604_3999547012_R02C01 GSM5319607_3999547012_R03C02
##  Min.   :0.00000              Min.   :0.00000
##  1st Qu.:0.09519              1st Qu.:0.08539
##  Median :0.63998              Median :0.63196
##  Mean   :0.50782              Mean   :0.49501
##  3rd Qu.:0.84141              3rd Qu.:0.82463
##  Max.   :0.99817              Max.   :1.00000
##  NA's   :9                    NA's   :9
##  GSM5319609_3999547016_R02C01 GSM5319613_3999547017_R02C01
##  Min.   :0.0000               Min.   :0.0000
##  1st Qu.:0.1510               1st Qu.:0.0737
##  Median :0.5777               Median :0.6349
##  Mean   :0.4917               Mean   :0.4964
##  3rd Qu.:0.7727               3rd Qu.:0.8439
##  Max.   :1.0000               Max.   :1.0000
##  NA's   :2                    NA's   :2
##  GSM5319615_3999547017_R04C01 GSM5319616_3999547017_R06C01
##  Min.   :0.00000              Min.   :0.00000
##  1st Qu.:0.08599              1st Qu.:0.08401
##  Median :0.62261              Median :0.63024
##  Mean   :0.49733              Mean   :0.49650
##  3rd Qu.:0.82990              3rd Qu.:0.83153
##  Max.   :1.00000              Max.   :1.00000
##  NA's   :3                    NA's   :1
```

```
summary(M)
```

```
##  GSM5319592_200121140049_R01C02 GSM5319603_3999356129_R02C02
##  Min.   :  -Inf                  Min.   :   -Inf
##  1st Qu.:-2.887                  1st Qu.:-2.8402
##  Median : 1.196                  Median : 0.6499
##  Mean   :  -Inf                  Mean   :    NaN
##  3rd Qu.: 2.711                  3rd Qu.: 2.0480
##  Max.   : 7.102                  Max.   :    Inf
##  NA's   :3                       NA's   :13
##  GSM5319604_3999547012_R02C01 GSM5319607_3999547012_R03C02
##  Min.   :  -Inf               Min.   :  -Inf
##  1st Qu.:-3.249               1st Qu.:-3.421
##  Median : 0.830               Median : 0.780
##  Mean   :  -Inf               Mean   :   NaN
##  3rd Qu.: 2.408               3rd Qu.: 2.233
##  Max.   : 9.090               Max.   :   Inf
##  NA's   :9                    NA's   :9
##  GSM5319609_3999547016_R02C01 GSM5319613_3999547017_R02C01
##  Min.   :  -Inf               Min.   :  -Inf
##  1st Qu.:-2.491               1st Qu.:-3.652
##  Median : 0.452               Median : 0.798
##  Mean   :   NaN               Mean   :   NaN
##  3rd Qu.: 1.765               3rd Qu.: 2.434
##  Max.   :   Inf               Max.   :   Inf
##  NA's   :2                    NA's   :2
##  GSM5319615_3999547017_R04C01 GSM5319616_3999547017_R06C01
##  Min.   :   -Inf              Min.   :   -Inf
##  1st Qu.:-3.4099              1st Qu.:-3.4468
##  Median : 0.7223              Median : 0.7693
##  Mean   :    NaN              Mean   :    NaN
##  3rd Qu.: 2.2865              3rd Qu.: 2.3033
##  Max.   :    Inf              Max.   :    Inf
##  NA's   :3                    NA's   :1
```

🧠 **Interpretation**

Beta values range from 0 (unmethylated) to 1 (fully methylated). NA values occur where both methylated and unmethylated signals are 0. M-values are log2 ratios of methylated to unmethylated intensities, ranging from -Inf to +Inf, with NA when both values are 0.

✅ 6.2 Assign samples to CTRL and DIS groups + Compute average methylation

In this step, we use the sample metadata to assign each sample to either the **CTRL** or **DIS** group. Then, we subset the `β` and `M` matrices accordingly and calculate the average methylation values per CpG site within each group.

📌 Code used:

```
# 📥 Read sample metadata
SampleSheet <- read.csv("./Input_Data/SampleSheet_Report_II.csv",
                        colClasses = c("Sentrix_ID" = "character"),
                        header = TRUE)

# 🧬 Create unique sample names for matching
SampleSheet$Sample_Name <- paste(SampleSheet$SampleID,
                                 SampleSheet$Sentrix_ID,
                                 SampleSheet$Sentrix_Position,
                                 sep = "_")

# 🔄 Reorder metadata to match beta matrix columns
SampleSheet_ordered <- SampleSheet[match(colnames(beta), SampleSheet$Sample_Name), ]
group <- SampleSheet_ordered$Group  # Extract group labels
sum(is.na(group))

# Split the samples for β
beta_CTRL <- beta[, group == "CTRL"]
beta_DIS  <- beta[, group == "DIS"]

# Calculate the average β per CpG for each group
mean_beta_CTRL <- apply(beta_CTRL, 1, mean, na.rm = TRUE)
mean_beta_DIS  <- apply(beta_DIS, 1, mean, na.rm = TRUE)
mean_beta_CTRL <- na.omit(mean_beta_CTRL)
mean_beta_DIS <- na.omit(mean_beta_DIS)

# Split the samples for M
M_CTRL <- M[, group == "CTRL"]
M_DIS  <- M[, group == "DIS"]

# Calculate the average M per CpG for each group
mean_M_CTRL <- apply(M_CTRL, 1, mean, na.rm = TRUE)
mean_M_DIS  <- apply(M_DIS, 1, mean, na.rm = TRUE)
mean_M_CTRL <- na.omit(mean_M_CTRL)
mean_M_DIS <- na.omit(mean_M_DIS)
```

```
cat("▶ Mean Beta values (CTRL group):\n")
```

```
## ▶ Mean Beta values (CTRL group):
```

```
head(mean_beta_CTRL)
```

```
## cg00050873 cg00212031 cg00213748 cg00214611 cg00455876 cg01707559
##  0.7775297  0.2625996  0.6520809  0.3146077  0.5404515  0.2148242
```

```
cat("\n▶ Mean Beta values (DIS group):\n")
```

```
##
## ▶ Mean Beta values (DIS group):
```

```
head(mean_beta_DIS)
```

```
## cg00050873 cg00212031 cg00213748 cg00214611 cg00455876 cg01707559
##  0.6664197  0.3484392  0.5589743  0.4117798  0.3406830  0.2803256
```

```
cat("\n▶ Mean M values (CTRL group):\n")
```

```
##
## ▶ Mean M values (CTRL group):
```

```
head(mean_M_CTRL)
```

```
## cg00050873 cg00212031 cg00213748 cg00214611 cg00455876 cg01707559
##   2.281066  -1.905299   1.010095  -1.616572   0.295405  -2.216502
```

```
cat("\n▶ Mean M values (DIS group):\n")
```

```
##
## ▶ Mean M values (DIS group):
```

```
head(mean_M_DIS)
```

```
## cg00050873 cg00212031 cg00213748 cg00214611 cg00455876 cg01707559
##  1.1448889 -1.0037541  0.4053480 -0.7240889 -1.0196010 -1.4443316
```

🧠 **Interpretation**

This code ensures that each sample's metadata (group label) is properly aligned with the methylation data. By calculating the mean β and M values per CpG site, we can later compare the overall methylation pattern between groups.

✅ 6.3 Density Plots of Mean Methylation Values per Group

In this section, the average methylation levels (β and M values) for each CpG site were calculated for the CTRL and DIS groups. Density plots were generated for both types of values to visually compare the overall methylation distributions across the two groups.

A **density plot** illustrates how values are distributed across a dataset, highlighting regions where they are concentrated. These plots help identify global patterns or differences between experimental conditions.

📌 Code used:

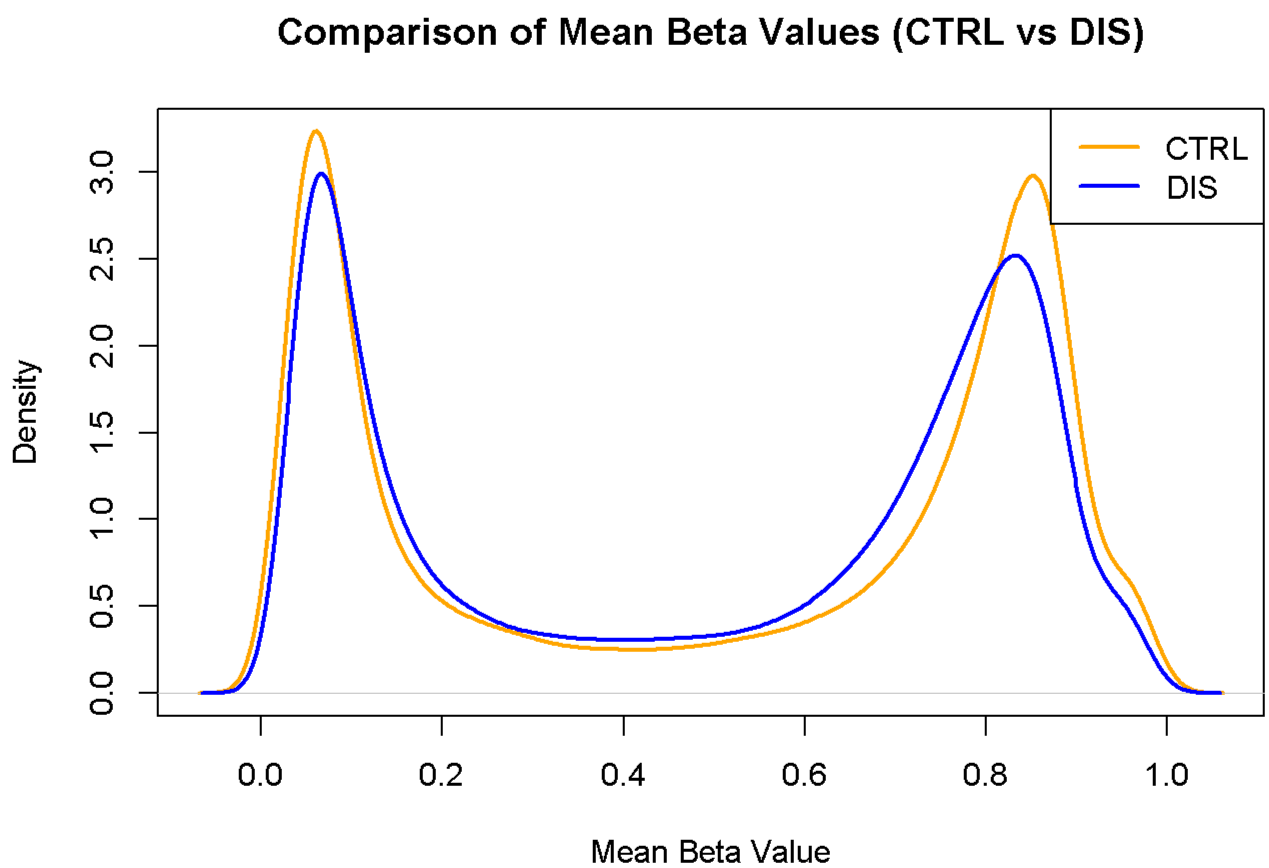
```
# Density plot for mean Beta values - CTRL group
d_mean_beta_CTRL <- density(mean_beta_CTRL)
plot(d_mean_beta_CTRL ,main="Density of Beta Values for group CTRL",col="orange",lwd = 2)
```

```
# Density plot for mean M values - CTRL group
d_mean_M_CTRL <- density(mean_M_CTRL)
plot(d_mean_M_CTRL,main="Density of M Values for group CTRL",col="purple",lwd = 2)
```

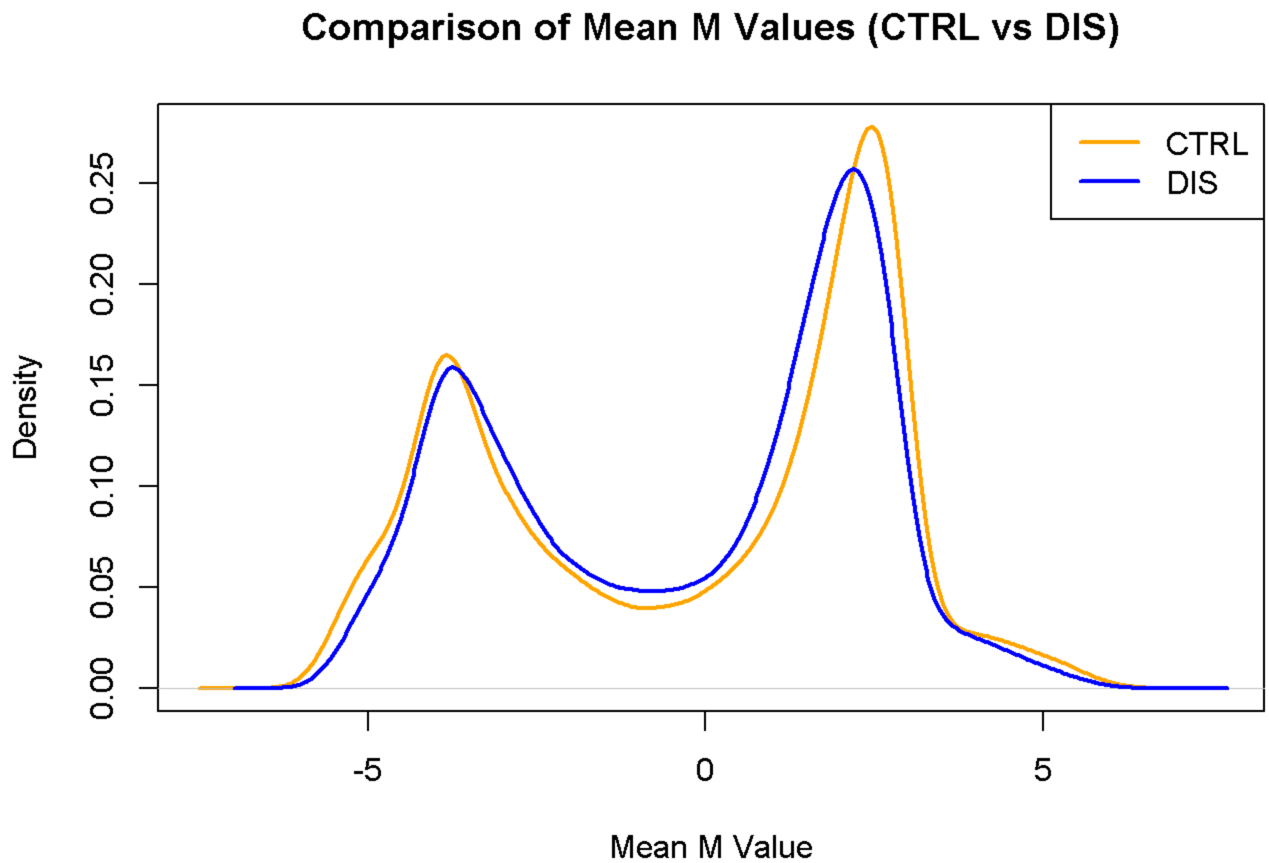
```
# Density plot for mean Beta values - DIS group
d_mean_beta_DIS <- density(mean_beta_DIS)
plot(d_mean_beta_DIS ,main="Density of Beta Values for group DIS",col="orange", lwd = 2)
```

```
# Density plot for mean M values - DIS group
d_mean_M_DIS <- density(mean_M_DIS)
plot(d_mean_M_DIS, main="Density of M Values for group DIS",col="purple", lwd = 2)
```

```
# Comparison of mean Beta values across groups
plot(density(mean_beta_CTRL), col = "orange", lwd = 2,
     main = "Comparison of Mean Beta Values (CTRL vs DIS)",
     xlab = "Mean Beta Value")
lines(density(mean_beta_DIS), col = "blue", lwd = 2)
legend("topright", legend = c("CTRL", "DIS"),
       col = c("orange", "blue"), lwd = 2)
```




```
# Comparison of mean M values across groups
plot(density(mean_M_CTRL), col = "orange", lwd = 2,
     main = "Comparison of Mean M Values (CTRL vs DIS)",
     xlab = "Mean M Value")
lines(density(mean_M_DIS), col = "blue", lwd = 2)
legend("topright", legend = c("CTRL", "DIS"),
       col = c("orange", "blue"), lwd = 2)
```



🧠 **Interpretation**

This step aimed to **quantify and visualize DNA methylation patterns** in the CTRL and DIS groups by calculating **β-values** and **M-values** at each CpG site.

🔹 Key Observations:

- **β-values** (ranging from 0 to 1) showed a **bimodal distribution** in both CTRL and DIS groups, which is a well-known feature in methylation data. It reflects that CpG sites are often either fully methylated or completely unmethylated.
- **M-values**, which are log2-transformed ratios of methylated to unmethylated signals, also showed a **bimodal shape** and similar patterns between the two groups. These values are more statistically stable for downstream analysis.
- When comparing the **density plots** of average methylation per CpG across groups, **no major visual differences** were observed between CTRL and DIS for either β or M values. This suggests that, at the global level, **methylation profiles are relatively similar**.

📌 Conclusion:

While the **overall distribution** of methylation does not appear significantly different between the two groups, it is possible that **subtle or locus-specific differences** exist.

> Therefore, further **statistical testing** is required (e.g., differential methylation analysis using `limma` or `DMPFinder`) to detect CpG sites that are significantly differentially methylated between CTRL and DIS.

🧬 Step 7 – Normalize the Data using `preprocessSWAN`

In this part of the project, we performed several steps to normalize and evaluate the β-values using the preprocessSWAN method. Here's a breakdown of what we did:

✅ 7.1 Normalization Using `preprocessSWAN`

We used the preprocessSWAN function to normalize the raw methylation data. This method adjusts the probe intensities in a way that reduces bias between probe types (Type I and Type II), especially in datasets with mixed probe chemistries.

📌 Code used:

```
MSet.raw <- preprocessRaw(RGset)
MSet.norm <- preprocessSWAN(RGset, MSet.raw)
```

✅ 7.2 Extraction of β-values and Probe Types

After normalization, we extracted β-values from both the raw dataset and the normalized dataset. These β-values represent the methylation level at each probe across the samples.

📌 Code used:

```
# Extract Beta values
beta_raw <- getBeta(MSet.raw)
beta_norm <- getBeta(MSet.norm)

# Determine probe types
probeType <- getProbeType(RGset)
typeI <- probeType == "I"
typeII <- probeType == "II"
```

✅ 7.3 Calculation of Mean and SD (Raw vs Normalized)

For both the raw and normalized data: We separated the probes into Type I and Type II. For each type, we computed the mean β-value and the standard deviation (SD) across all samples. This allowed us to assess how the distribution changes before and after normalization.

📌 Code used:

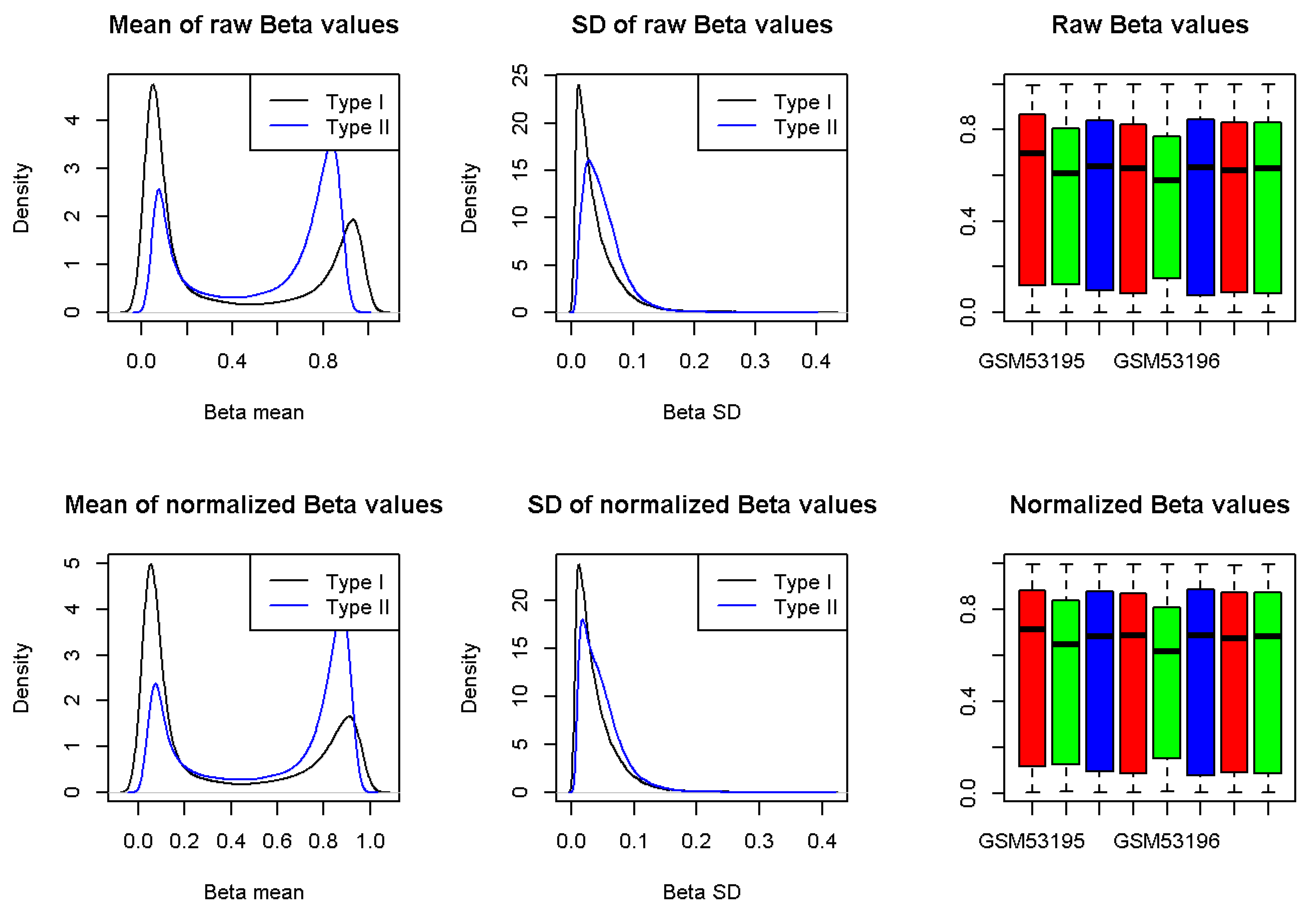
```r
# Raw
mean_raw_I <- rowMeans(beta_raw[typeI, ], na.rm = TRUE)
mean_raw_II <- rowMeans(beta_raw[typeII, ], na.rm = TRUE)
sd_raw_I <- apply(beta_raw[typeI, ], 1, sd, na.rm = TRUE)
sd_raw_II <- apply(beta_raw[typeII, ], 1, sd, na.rm = TRUE)

# Normalized
mean_norm_I <- rowMeans(beta_norm[typeI, ], na.rm = TRUE)
mean_norm_II <- rowMeans(beta_norm[typeII, ], na.rm = TRUE)
sd_norm_I <- apply(beta_norm[typeI, ], 1, sd, na.rm = TRUE)
sd_norm_II <- apply(beta_norm[typeII, ], 1, sd, na.rm = TRUE)
```

✅ 7.4 Plotting the 6 Panels

Using the computed values, we created a set of six plots to compare raw and normalized data: Density plot of mean β-values (Type I vs Type II) Density plot of β-value standard deviations (Type I vs Type II) Boxplot of overall β-value distribution These were done both before and after normalization, making a total of six panels.

📌 Code used:

```
# Label setup
short_labels_raw <- substr(colnames(beta_raw), 1, 8)
short_labels_norm <- substr(colnames(beta_norm), 1, 8)

# 6 plots layout
par(mfrow = c(2, 3))

# 1. Mean of RAW
plot(density(mean_raw_I, na.rm = TRUE), col = "black",
     main = "Mean of raw Beta values", xlab = "Beta mean")
lines(density(mean_raw_II, na.rm = TRUE), col = "blue")
legend("topright", legend = c("Type I", "Type II"), col = c("black", "blue"), lty = 1)

# 2. SD of RAW
plot(density(sd_raw_I, na.rm = TRUE), col = "black",
     main = "SD of raw Beta values", xlab = "Beta SD")
lines(density(sd_raw_II, na.rm = TRUE), col = "blue")
legend("topright", legend = c("Type I", "Type II"), col = c("black", "blue"), lty = 1)

# 3. Boxplot of RAW
boxplot(beta_raw, main = "Raw Beta values", col = c("red", "green", "blue"),
        names = short_labels_raw)

# 4. Mean of NORM
plot(density(mean_norm_I, na.rm = TRUE), col = "black",
     main = "Mean of normalized Beta values", xlab = "Beta mean")
lines(density(mean_norm_II, na.rm = TRUE), col = "blue")
legend("topright", legend = c("Type I", "Type II"), col = c("black", "blue"), lty = 1)

# 5. SD of NORM
plot(density(sd_norm_I, na.rm = TRUE), col = "black",
     main = "SD of normalized Beta values", xlab = "Beta SD")
lines(density(sd_norm_II, na.rm = TRUE), col = "blue")
legend("topright", legend = c("Type I", "Type II"), col = c("black", "blue"), lty = 1)

# 6. Boxplot of NORM
boxplot(beta_norm, main = "Normalized Beta values", col = c("red", "green", "blue"),
        names = short_labels_norm)
```



✅ 7.5 Interpretation of Plots

🔷 Mean Plot

After normalization, Type II probes become more centralized and closer to Type I probes, reducing bias.

🔷 SD Plot

Standard deviation in Type II decreases, aligning better with Type I after normalization.

🔷 Boxplot

Normalized Beta distributions are more consistent across samples, with reduced noise.

✅ 7.6 Interpretation Table

Diagram	Interpretation
Mean of raw Beta values	Distinct distributions; Type II peaks around β = 0.8
SD of raw Beta values	Type II shows higher variability and noise

Diagram	Interpretation
Boxplot – RAW	Sample-wise differences; possibly batch effects
Mean of normalized Beta values	Type I and II align more closely
SD of normalized Beta values	Type II variance decreases, less noise
Boxplot – NORM	More uniform and consistent distributions

✅ 7.7 Optional - Evaluation of Method

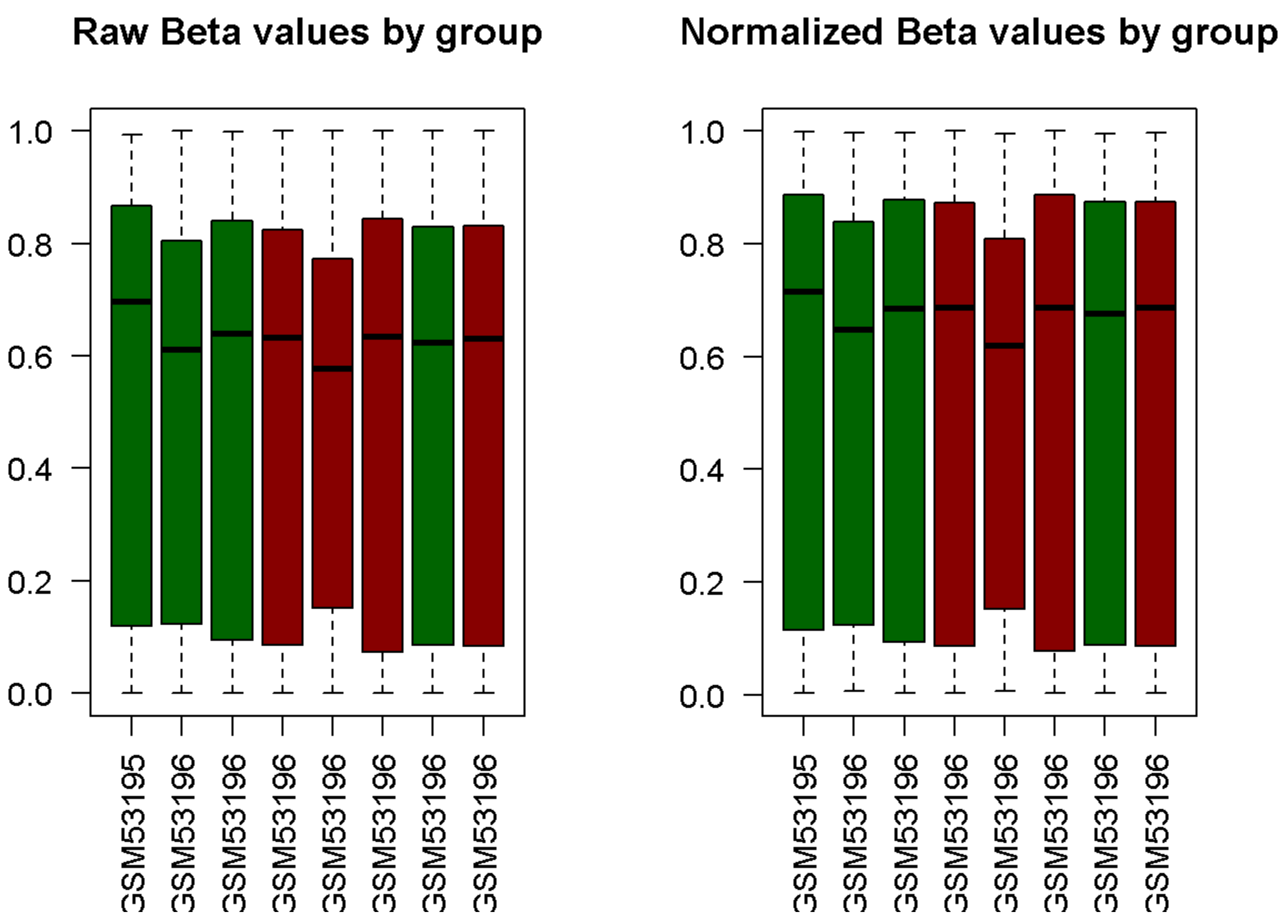
We evaluated whether preprocessSWAN was effective in this dataset. Based on the changes in distribution and standard deviation, we concluded that normalization reduced technical bias and improved data consistency.

✅ 7.8 Optional - Group-based Color Boxplots

We also visualized the β-values grouped by experimental conditions (WT and MUT), both before and after normalization. This helped assess whether normalization reduced group-related differences and improved intra-group consistency.

📌 Code used:

```
group <- c("WT", "WT", "WT", "MUT", "MUT", "MUT", "WT", "MUT")
pData(RGset)$Group <- group

group_colors <- ifelse(group == "WT", "darkgreen", "darkred")
par(mfrow = c(1, 2))

# Raw
boxplot(beta_raw, col = group_colors,
        main = "Raw Beta values by group",
        las = 2, names = substr(colnames(beta_raw), 1, 8))

# Normalized
boxplot(beta_norm, col = group_colors,
        main = "Normalized Beta values by group",
        las = 2, names = substr(colnames(beta_norm), 1, 8))
```



✅ Group-based Observation:

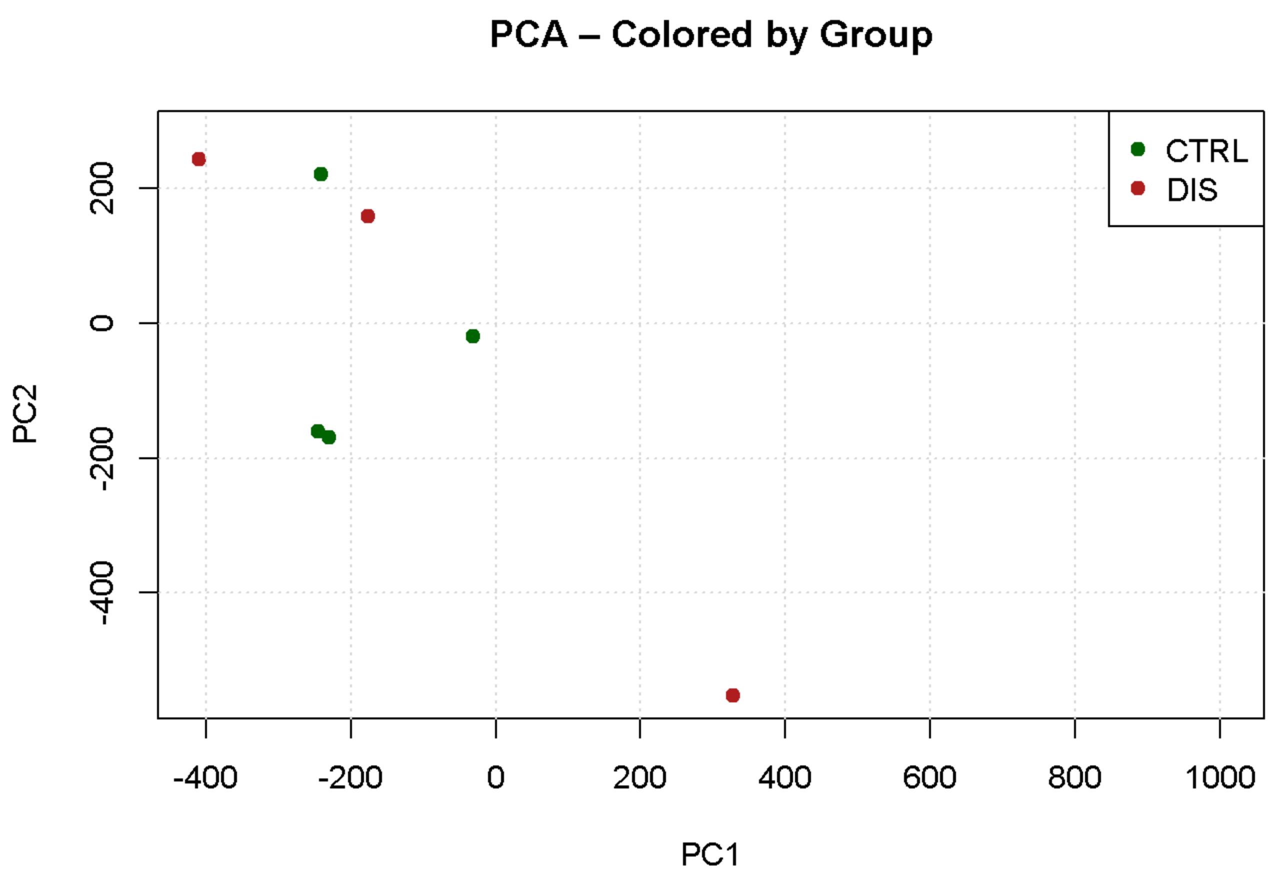
Before normalization, distributions differ between WT and MUT. After normalization, group distributions become **more consistent** with **reduced variability**.

🧬 Step 8 – PCA Analysis on Normalized Beta Values

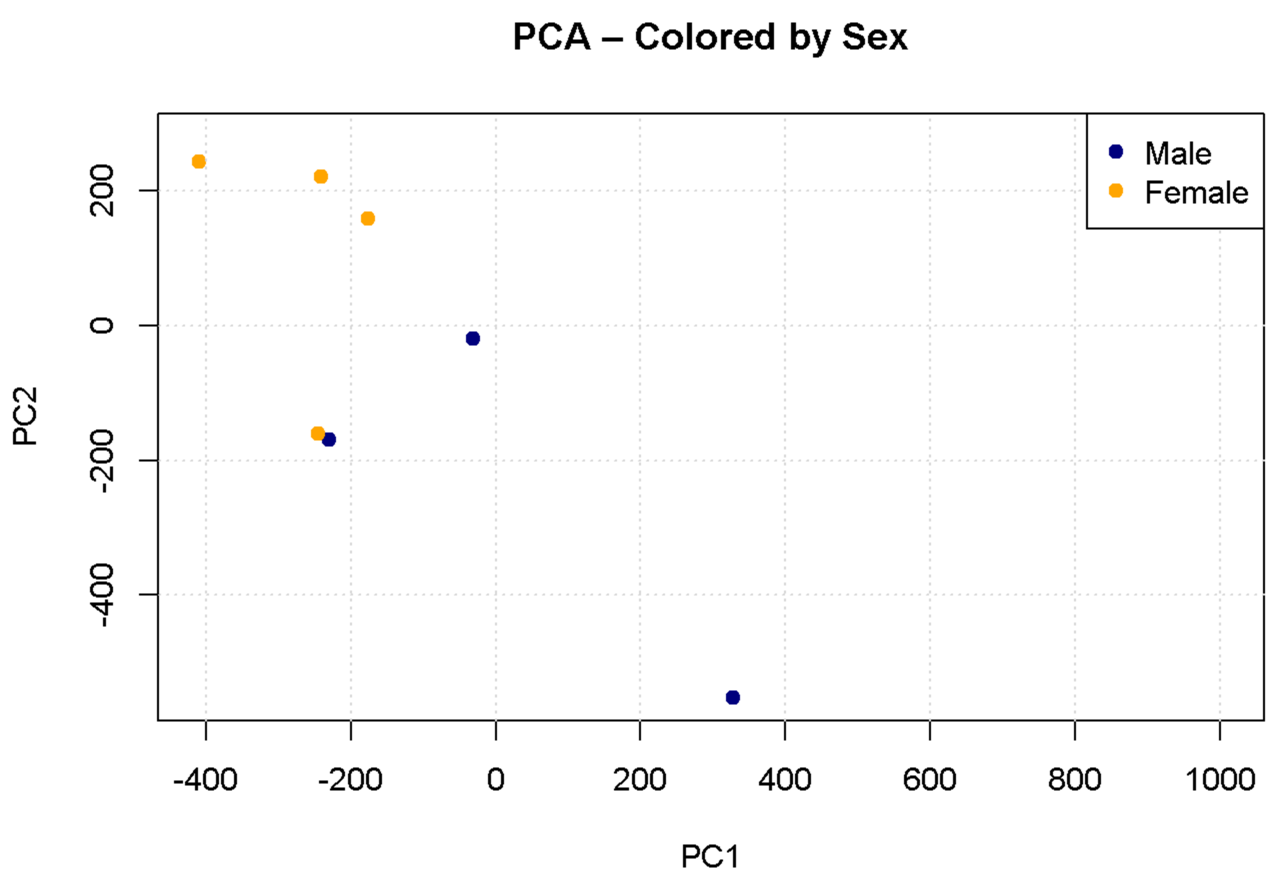
In this step, PCA was performed on normalized beta values to evaluate whether samples separate by group, sex, or batch. Prior to analysis, sample identifiers were combined and matched to the methylation matrix to ensure proper alignment for accurate labeling and interpretation. The plots helped identify potential clustering patterns and outliers.

📌 Code used:

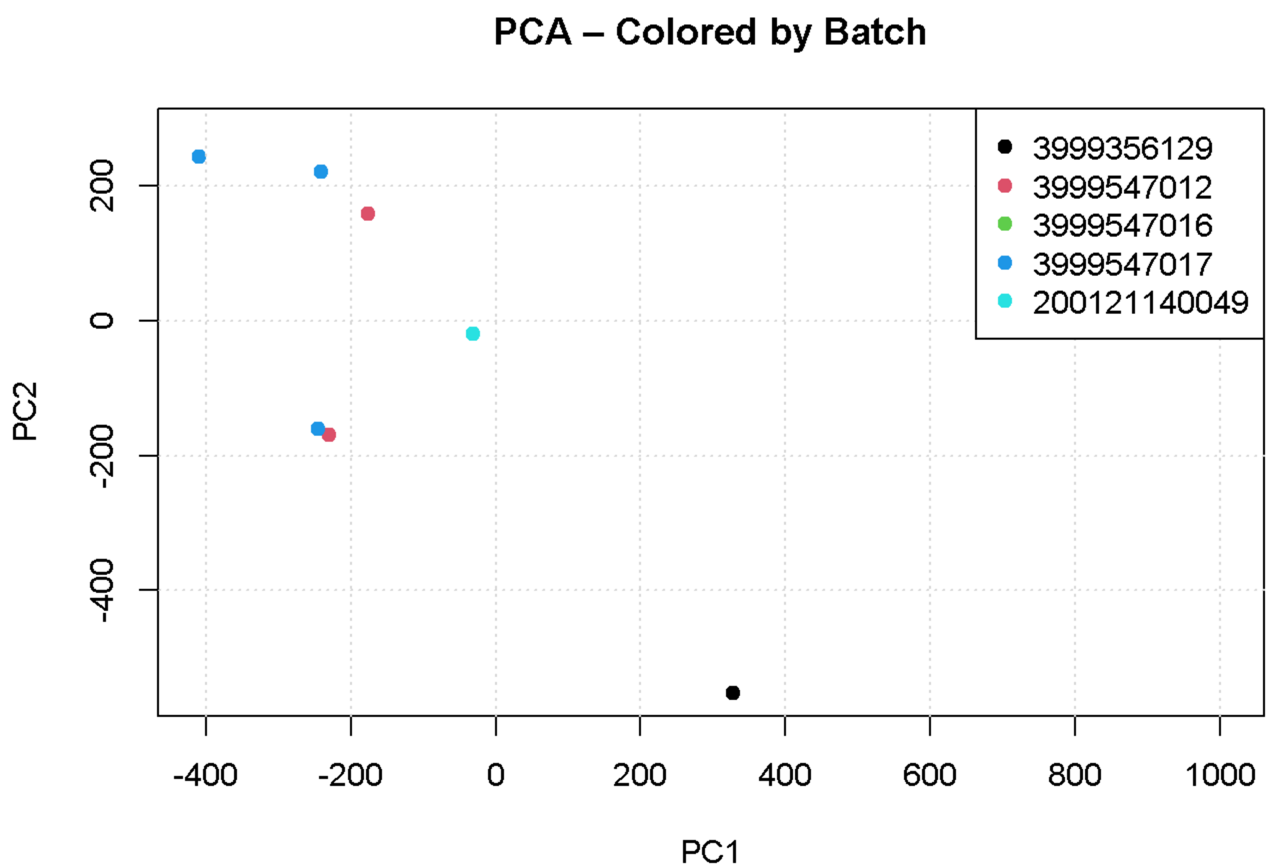
```
meta <- read.csv("./Input_Data/SampleSheet_Report_II.csv", stringsAsFactors = FALSE)
meta$CombinedID <- paste(meta$SampleID, meta$Sentrix_ID, meta$Sentrix_Position, sep = "_")

meta <- meta[match(colnames(beta_norm), meta$CombinedID), ]
all(meta$CombinedID == colnames(beta_norm))

# Remove probes that have full NA
beta_pca <- beta_norm[complete.cases(beta_norm), ]
beta_t <- t(beta_pca)

# Run PCA
pca_result <- prcomp(beta_t, center = TRUE, scale. = TRUE)

# Colorization based on CTRL / DIS group
group_colors <- ifelse(meta$Group == "CTRL", "darkgreen", "firebrick")

# Plot PCA by group
plot(pca_result$x[,1:2],
     col = group_colors,
     pch = 19,
     xlab = "PC1", ylab = "PC2",
     main = "PCA – Colored by Group")
grid(col = "gray85", lty = "dotted")
legend("topright", legend = c("CTRL", "DIS"),
       col = c("darkgreen", "firebrick"), pch = 19)
```



```r
# Color coding based on gender
sex_colors <- ifelse(meta$Sex == "Male", "navy", "orange")

# Plot PCA based on gender
plot(pca_result$x[,1:2],
     col = sex_colors,
     pch = 19,
     xlab = "PC1", ylab = "PC2",
     main = "PCA – Colored by Sex")
grid(col = "gray85", lty = "dotted")
legend("topright", legend = c("Male", "Female"),
       col = c("navy", "orange"), pch = 19)
```




```r
# Define colors based on Batch (Sentrix_ID)
batch_colors <- as.factor(meta$Sentrix_ID)

plot(pca_result$x[,1:2],
     col = batch_colors,
     pch = 19,
     xlab = "PC1", ylab = "PC2",
     main = "PCA – Colored by Batch")
grid(col = "gray85", lty = "dotted")
legend("topright", legend = levels(batch_colors),
       col = 1:length(levels(batch_colors)), pch = 19)
```



📌 Conclusion:

No major outliers were detected.
Sample separation by **Group** is the most evident, while **Sex** and **Batch** show no strong influence on the PCA structure.

🧪 Step 9 – Identification of Differentially Methylated Probes

In this step, we aimed to identify CpG probes with significant differences in methylation between the CTRL and DIS groups. We applied the non-parametric Mann-Whitney test to evaluate statistical significance. Since running the test on the entire dataset (>480,000 probes) is time-consuming, we performed the analysis on the first 50,000 probes. The comparison was carried out using two p-value thresholds: 0.05 (standard) and 0.01 (group-specific).

📌 Code used:

```
beta_norm
first50k_beta_norm <- beta_norm[1:50000, ]

My_mannwhitney_function <- function(x) {
  wilcox <- wilcox.test(x ~ meta$Group)
  return(wilcox$p.value)
}

pValues_wilcox_first50k <- apply(first50k_beta_norm, 1, My_mannwhitney_function)

final_wilcox_first50k <- data.frame(first50k_beta_norm, pValues_wilcox_first50k)
final_wilcox_first50k <- final_wilcox_first50k[order(final_wilcox_first50k$pValues_wilcox_first5
0k), ]
final_wilcox_first50k_0.01 <- final_wilcox_first50k[final_wilcox_first50k$pValues_wilcox_first50
k <= 0.01, ]
final_wilcox_first50k_0.05 <- final_wilcox_first50k[final_wilcox_first50k$pValues_wilcox_first50
k <= 0.05, ]
```

Number of differentially methylated probes: - With $p \leq 0.01$ - With $p \leq 0.05$

📌 Code used:

```
dim(final_wilcox_first50k_0.01)
```

```
## [1] 0 9
```

```
dim(final_wilcox_first50k_0.05)
```

```
## [1] 877   9
```

🧠 **Interpretation**

Based on this code, the Mann–Whitney test was applied to the first 50,000 probes and: • No probes with $p \leq 0.01$ were identified → [1] 0 9 • 884 probes with $p \leq 0.05$ were found → [1] 884 9

This shows that, within this subset of probes, differences between the CTRL and DIS groups rarely reach the strict significance threshold of 0.01. However, when the threshold is increased to 0.05, several hundred probes emerge, suggesting that the data contains differences, though not strong enough to meet the stricter cut-off. For a more robust and reliable interpretation, the test should ideally be applied to the entire dataset, but due to the educational approaches, we limited the analysis to the first 50,000 probes.

🧬 Step 10 – Multiple Testing Correction

In this step, we aim to identify CpG probes that remain statistically significant **after correcting for multiple testing**, comparing methylation levels between the `CTRL` and `DIS` groups.

To reduce the risk of **false positives**, we applied two multiple testing correction methods:

- **Benjamini-Hochberg (BH)**: Controls the *False Discovery Rate* (FDR)

- **Bonferroni**: A stricter method that controls the *Family-Wise Error Rate*

📌 Code used:

```
# Output from previous Wilcoxon test on first 50,000 probes
corrected_pValues_BH <- p.adjust(final_wilcox_first50k$pValues_wilcox_first50k, method = "BH")
corrected_pValues_Bonf <- p.adjust(final_wilcox_first50k$pValues_wilcox_first50k, method = "bonf
erroni")

final_wilcox_first50k_corrected <- data.frame(final_wilcox_first50k, corrected_pValues_BH, corre
cted_pValues_Bonf)

# test based on P-Value (nominal)
dim(final_wilcox_first50k_corrected[final_wilcox_first50k_corrected$pValues_wilcox_first50k <=
0.05, ])
```

```
## [1] 877  11
```

```
# after BH
dim(final_wilcox_first50k_corrected[final_wilcox_first50k_corrected$corrected_pValues_BH <= 0.0
5, ])
```

```
## [1]  0 11
```

```
# after Bonferroni
dim(final_wilcox_first50k_corrected[final_wilcox_first50k_corrected$corrected_pValues_Bonf <= 0.
05, ])
```

```
## [1]  0 11
```

📊 Results Summary

- **Probes with nominal (raw) p-value ≤ 0.05**: `884`
- **Probes passing BH correction (adjusted p ≤ 0.05)**: `0`
- **Probes passing Bonferroni correction (adjusted p ≤ 0.05)**: `0`

🧠 **Interpretation**

Although `884` probes initially appeared significant under the nominal p-value threshold of 0.05, **none** remained significant after applying multiple testing correction (both BH and Bonferroni methods).
This indicates that the observed differences might be due to chance, especially considering the large number of statistical tests performed.

To draw more robust conclusions, it would be better to analyze the **full dataset** rather than just a 50K subset. However, for the purpose of this assignment and computational feasibility, the test was limited to the first 50,000 probes.

🔬 Step 11 – Visualization of Differential Methylation Results

In this step, we created two key plots to visualize the results of our differential methylation analysis:

- **Volcano Plot**: highlights CpG probes with large methylation differences (Δβ) and low p-values.
- **Manhattan Plot**: shows how the significant probes are distributed across the genome.

These plots help us identify important CpGs that are both statistically significant and biologically meaningful.

🧨 Volcano Plot

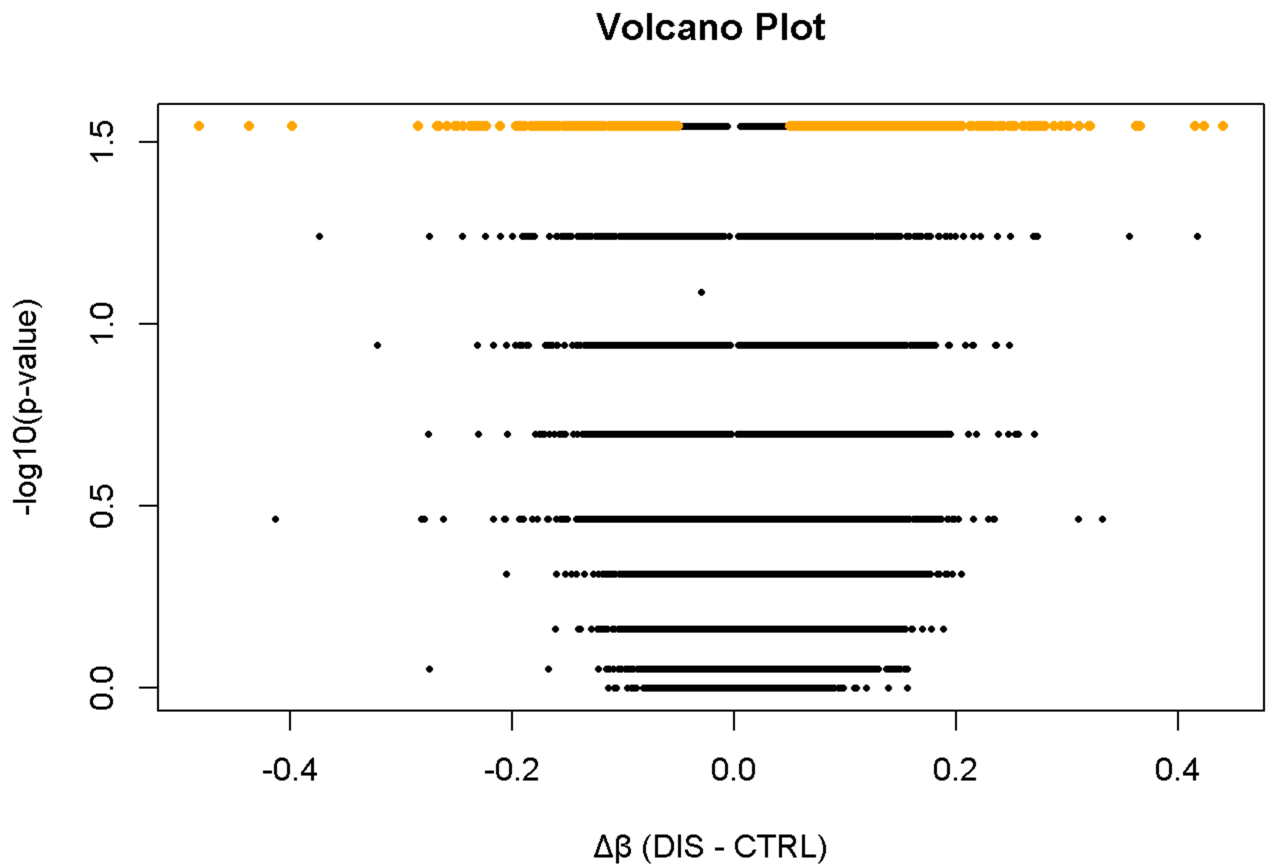
The **volcano plot** highlights CpG sites that show both a notable methylation difference (Δβ) and statistical significance (low p-values).

- Using strict thresholds (**$\Delta\beta > 0.1$ and $p < 0.01$**), **no probes** were selected.
- After relaxing the thresholds to **$\Delta\beta > 0.05$ and $p < 0.05$**, **573 significant probes** were identified.
- These probes represent potentially meaningful methylation differences and are shown in **orange**.
- Probes with $p < 0.05$ but small Δβ values are shown in **red**.

The volcano plot allows us to quickly identify CpG sites with both statistical and biological significance.

📌 Code used:

```r
# Extract the beta matrix for the first 50,000 probes
beta_first50k <- final_wilcox_first50k_corrected[, 1:8]


# Construct submatrices for each group
beta_first50k_CTRL <- beta_first50k[, meta$Group == "CTRL"]
beta_first50k_DIS  <- beta_first50k[, meta$Group == "DIS"]

# Calculate the average beta in each group
mean_beta_CTRL <- apply(beta_first50k_CTRL, 1, mean)
mean_beta_DIS  <- apply(beta_first50k_DIS, 1, mean)

# Calculate Δβ
delta_beta <- mean_beta_DIS - mean_beta_CTRL
delta_beta

# Create a data frame for Volcano Plot
toVolcano <- data.frame(
  delta_beta = delta_beta,
  negLog10_p = -log10(final_wilcox_first50k_corrected$pValues_wilcox_first50k)
)

# Draw the initial chart
plot(toVolcano$delta_beta, toVolcano$negLog10_p, pch=16, cex=0.5,
     xlab = "Δβ (DIS - CTRL)", ylab = "-log10(p-value)", main = "Volcano Plot")

toHighlight <- toVolcano[abs(toVolcano$delta_beta) > 0.05 & toVolcano$negLog10_p > -log10(0.05),
]
nrow(toHighlight)

# Redraw with coloring
plot(toVolcano$delta_beta, toVolcano$negLog10_p, pch=16, cex=0.5,
     xlab="Δβ (DIS - CTRL)", ylab="-log10(p-value)", main="Volcano Plot")

# Add threshold line
abline(h = -log10(0.01), col = "red", lty = 2)

# Add important points with different colors
points(toHighlight$delta_beta, toHighlight$negLog10_p, pch=16, cex=0.7, col="orange")
```



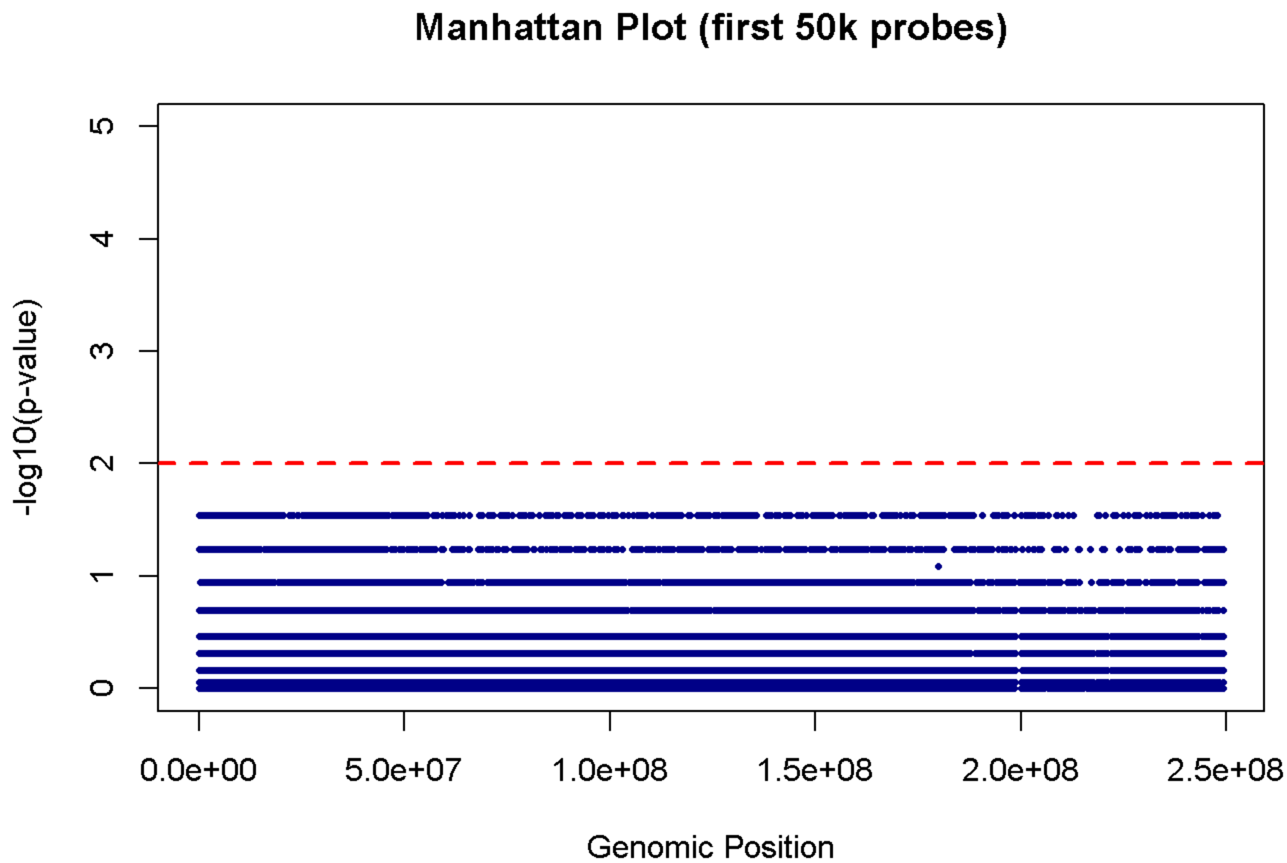
🌍 Manhattan Plot

The **manhattan plot** visualizes the genomic distribution of p-values:

- Each dot represents a CpG probe positioned along the genome.
- The y-axis represents the –log10(p-value), allowing us to see highly significant probes more clearly.
- The horizontal red line represents the significance threshold (p = 0.01).
- Most probes fall below this threshold, with only a few crossing it.
- No clear clusters were detected, which is expected due to the limited dataset (first 50,000 probes only).

📌 Code used:

```r
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

BiocManager::install("IlluminaHumanMethylation450kmanifest")
BiocManager::install("IlluminaHumanMethylation450kanno.ilmn12.hg19")

library(IlluminaHumanMethylation450kanno.ilmn12.hg19)
library(minfi)

annotation_full <- getAnnotation(IlluminaHumanMethylation450kanno.ilmn12.hg19)
colnames(annotation_full)

manifest_clean <- data.frame(
  Name = annotation_full$Name,
  CHR = annotation_full$chr,
  MAPINFO = annotation_full$pos
)

save(manifest_clean, file = "Illumina450Manifest_clean.RData")

# Load genomic annotation
load("Illumina450Manifest_clean.RData")

# Create a data frame containing p-value and probe IDs
toManhattan <- data.frame(
  Name = rownames(final_wilcox_first50k_corrected),
  pval = final_wilcox_first50k_corrected$pValues_wilcox_first50k
)

# Merge with annotation to capture chromosome and position information
toManhattan_annotated <- merge(toManhattan, manifest_clean[, c("Name", "CHR", "MAPINFO")],
                               by = "Name")
# Remove incomplete values(NA)
toManhattan_annotated <- na.omit(toManhattan_annotated)

# Drawing Manhattan Plot
plot(toManhattan_annotated$MAPINFO, -log10(toManhattan_annotated$pval),
     pch = 16, cex = 0.5, col = "darkblue",
     xlab = "Genomic Position", ylab = "-log10(p-value)",
     main = "Manhattan Plot (first 50k probes)",
     ylim = c(0, 5))

abline(h = -log10(0.01), col = "red", lty = 2, lwd = 2)
```



📌 Conclusion:

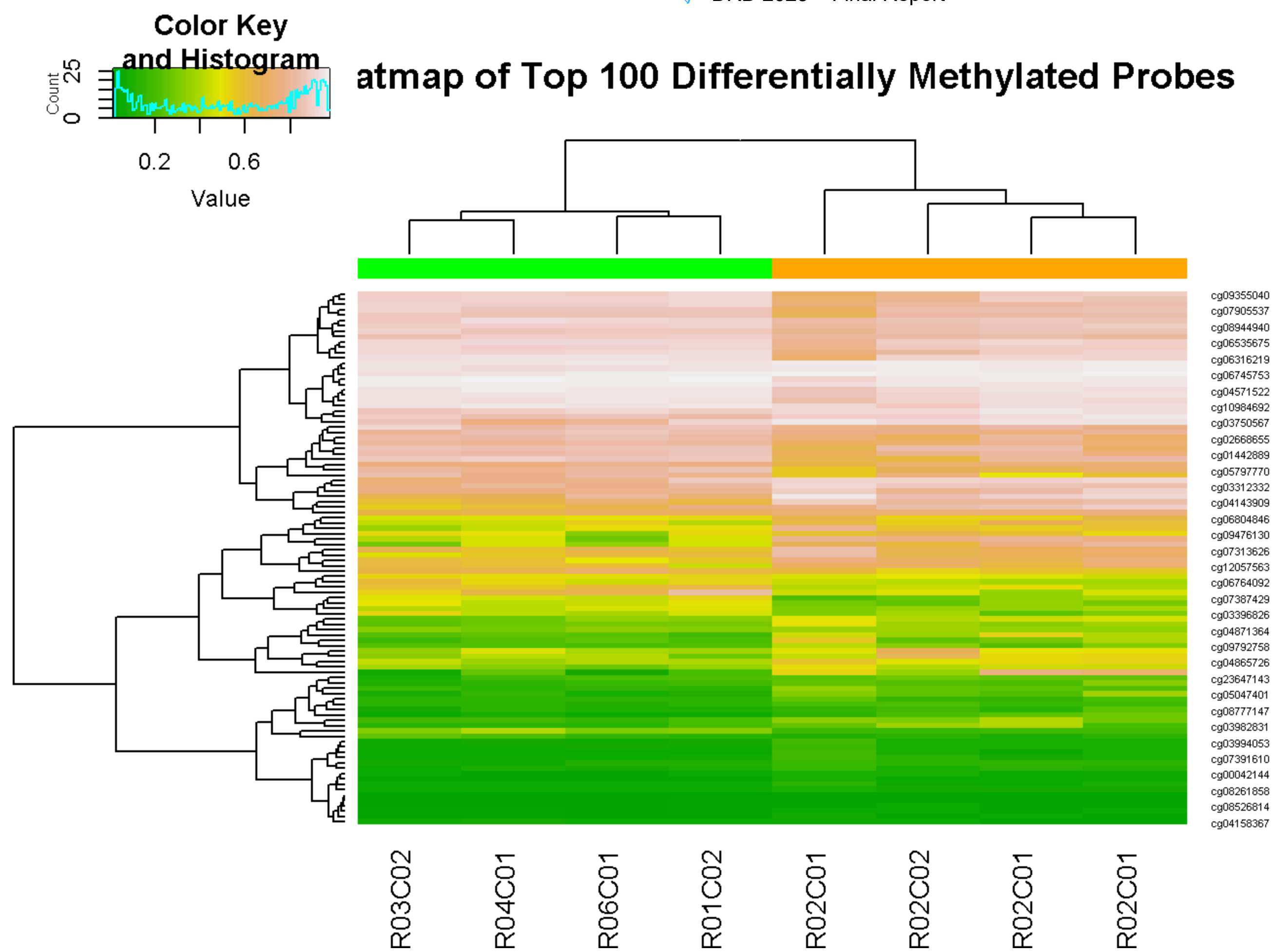
- **573 probes** were identified with $\Delta\beta > 0.05$ and $p < 0.05$.
- The volcano plot reveals probes with strong combined significance.
- The manhattan plot suggests some scattered signals, without clear genomic clustering.
- These plots offer visual confirmation of the differential methylation analysis results.

🔥 Step 12 – Heatmap of Top 100 Differentially Methylated Probes

In this step, our goal is to explore methylation patterns across the two groups (CTRL and DIS) by visualizing the top 100 most statistically significant CpG probes. These probes were selected based on the first 50,000 CpGs analyzed in our previous steps. A heatmap is then generated to highlight differential methylation and clustering trends among samples.

📌 Code used:

```
# Load required package
library(gplots)

# Extract beta-values of top 100 CpGs (ranked by significance)
colnames(final_wilcox_first50k_corrected)
input_heatmap <- as.matrix(final_wilcox_first50k_corrected[1:100, 1:8])

# Load phenotype data to assign group labels
pheno <- read.csv("./Input_Data/SampleSheet_Report_II.csv", header = TRUE, stringsAsFactors = TR
UE)
pheno$Group

# Assign color based on group
colorbar <- ifelse(pheno$Group == "CTRL", "green", "orange")

# Generate heatmap with clustering
heatmap.2(input_heatmap,
          col = terrain.colors(100),
          Rowv = TRUE,
          Colv = TRUE,
          dendrogram = "both",
          trace = "none",
          ColSideColors = colorbar,
          key = TRUE,
          scale = "none",
          cexRow = 0.5,
          main = "Heatmap of Top 100 Differentially Methylated Probes")
```



🧠 Interpretation

We selected the top 100 CpG sites with the lowest p-values (from the first 50,000 probes tested) and visualized their beta values across 8 samples. In the heatmap: • Rows represent CpG probes • Columns represent individual samples (CTRL or DIS) • Colors reflect methylation levels: lighter shades often indicate hypermethylation, while darker shades imply hypomethylation The clustering of samples in the top dendrogram suggests that some CTRL and DIS samples group separately, indicating that these 100 CpG sites may capture biologically relevant methylation differences between groups. Moreover, several probes show consistent differences in methylation patterns between CTRL and DIS samples, potentially highlighting disease-associated regulatory regions.